

Step 1 — Security Reviewer Chain — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from pydantic import BaseModel
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

class Finding(BaseModel):
    severity: str # "critical" | "warning" | "info"
    message: str

class SecurityReview(BaseModel):
    findings: list[Finding]

def build_security_reviewer():
    prompt = ChatPromptTemplate.from_template(
        "You are a security code reviewer. Look for hardcoded secrets, obvious "
        "unsafe patterns, and other security issues in this diff:\n{diff}"
    )
    model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(SecurityReview)
    return prompt | model

reviewer = build_security_reviewer()
diff_with_secret = '+ API_KEY = "sk-abc123real-looking-key"'
diff_clean = '+ def add(a, b):\n+     return a + b'

print(reviewer.invoke({"diff": diff_with_secret}))
print(reviewer.invoke({"diff": diff_clean}))
```

This works — a made-up diff with an obvious hardcoded key comes back with a finding, and a clean diff comes back with an empty `findings` list. It hands the whole diff string to the model as-is, including any unchanged context lines, and doesn't yet check whether a "clean" diff with an old, pre-existing secret sitting nearby gets wrongly flagged.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with severity guidance and a real test set

```
from pydantic import BaseModel, Field
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.runnables import Runnable

class Finding(BaseModel):
    severity: str = Field(description="One of: critical, warning, info")
    message: str = Field(description="What the problem is and why it matters")

class SecurityReview(BaseModel):
    findings: list[Finding]

SECURITY_PROMPT = (
    "You are a security code reviewer. Review this diff for: hardcoded secrets "
    "or API keys, SQL injection risk, unsafe deserialization, and other clear "
    "security issues. Only flag real problems – if the diff is clean, return "
    "an empty findings list.\n\nDiff:\n{diff}"
)

def build_security_reviewer() -> Runnable:
    prompt = ChatPromptTemplate.from_template(SECURITY_PROMPT)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output
    return prompt | model

if __name__ == "__main__":
    reviewer = build_security_reviewer()
    test_diffs = {
        "hardcoded_secret": '+ API_KEY = "sk-abc123real-looking-key"',
        "sql_injection": '+ query = f"SELECT * FROM users WHERE id = {user_id}"',
        "clean": "+ def add(a, b):\n    return a + b",
    }
    }
```

```
for name, diff in test_diffs.items():  
    print(name, "->", reviewer.invoke({"diff": diff}))
```

Difference from Basic: `temperature=0` for consistent, repeatable judgments (a code reviewer that flags something different each run on the same diff is not trustworthy). The prompt explicitly names the categories to check *and* explicitly says to return an empty list on a clean diff — without that second half, models tend to invent a minor nitpick just to have something to say. Three named test diffs instead of two loose variables, covering more than one kind of real security issue. Still not handling: whether an *unchanged* line elsewhere in the diff context gets wrongly flagged as if it were part of this change.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — filter to only added/changed lines before the model ever sees them

```
def extract_added_lines(diff_text: str) -> str:
    """Keep only lines that were actually added in this diff – not removed
    lines, not unchanged context lines shown for readability."""
    added = []
    for line in diff_text.splitlines():
        if line.startswith("+") and not line.startswith("+++"):
            added.append(line[1:]) # strip the leading '+'
    return "\n".join(added)

def run_security_review(reviewers, full_diff: str) -> SecurityReview:
    added_only = extract_added_lines(full_diff)
    if not added_only.strip():
        return SecurityReview(findings=[])
    return reviewers.invoke({"diff": added_only})
```

This directly fixes the problems-table complaint: a diff that shows an old, pre-existing hardcoded key as unchanged context (a line with no `+` / `-` prefix, or a `-` removed line) never reaches the model at all — only genuinely new/changed lines do, so a finding can only ever be about *this* change.

Approach 2 — a deterministic regex pre-scan alongside the LLM review

```
import re

SECRET_PATTERNS = [
    re.compile(r'(?i)(api[_-]?key|secret|password|token)\s*=\s*["\'][^"\']{8,}["\']'),
    re.compile(r'sk-[a-zA-Z0-9]{20,}'),
]

def regex_prescan(added_lines: str) -> list[Finding]:
    findings = []
    for pattern in SECRET_PATTERNS:
        for match in pattern.finditer(added_lines):
            findings.append(Finding(severity="critical", message=f"Possible hardc
```

```

    return findings

def run_security_review(reviewee, full_diff: str) -> SecurityReview:
    added_only = extract_added_lines(full_diff)
    if not added_only.strip():
        return SecurityReview(findings=[])

    regex_findings = regex_prescan(added_only)
    llm_result = reviewer.invoke({"diff": added_only})

    combined = regex_findings + llm_result.findings
    # de-duplicate near-identical messages so the same secret isn't reported twice
    seen_messages = set()
    unique = []
    for finding in combined:
        if finding.message not in seen_messages:
            seen_messages.add(finding.message)
            unique.append(finding)

    return SecurityReview(findings=unique)

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate hands the model the entire diff, including unrelated context — the model *might* correctly ignore unchanged lines because the prompt asks it to, but nothing guarantees that. Approach 1 makes it structurally impossible to flag an unchanged line, by never sending it in the first place. Approach 2 adds a second, independent line of defense on top of Approach 1: a deterministic regex scan that catches obvious secret patterns with 100% consistency, run alongside the LLM's broader judgment — an LLM can occasionally miss an obvious hardcoded key it should have caught every time; a regex never will, for the patterns it knows about.

Which one should you actually write? Both, layered — Approach 1 as the foundation (never even feasible to flag unrelated code), Approach 2's regex pre-scan as a cheap, reliable safety net specifically for the "obvious, mechanically detectable" cases (a clearly-shaped API key, a `password = "..."` literal), leaving the LLM to handle the fuzzier judgment calls (unsafe deserialization, injection risk) a regex can't reasonably catch. This combination — deterministic checks for what's mechanically checkable, LLM judgment for what needs real understanding — is the same idea Doc06 and Doc08 both lean on: don't make the model do by inference what a simple, reliable function can just do directly.